Load Bounds: Node, Edge and Lane Accounting for a Rings Node under Free Identities

Ryan J. Kung

Abstract

A Rings identity is a key digest, so anyone can mint as many as they like. Every table a remote party can grow must therefore be accounted to something that is not free. We take the authenticated neighbour edge: a connection the node itself admitted, of which it holds at most $\mathcal{R}$ at once. One accounting law covers every such table: each current edge has a reserved floor, overflow and the charges of departed edges share a pool, and the node-wide bound is the sum. An honest edge is then refused only past its own floor with the pool full, and a free identity consumes at most the share of the edge that carries it. We apply the law to the receiver's replay streams (with an eviction horizon that cannot reopen replay), to the removal store, to received frames through a credit-reclaim protocol that makes the frame bound hard and independent of the connection count, and to reassembled messages, which are charged once at their first chunk so that no honest message is refused. We state the complexity obligations of the admission hot paths, and show that the barrier's control exemption is the identity "a lane waits for reassembly iff it carries chunks".

I. Problem

#907
: The receiver's replay table never evicts, and a new stream fails closed once the table is full. Streams are keyed by the origin account, which costs nothing, so 16 384 key pairs and one message each lock a node out of every new origin, permanently.

#1060, residual
: The removal store charges each sender a share, and senders are free keys, so 64 identities fill it.

#934
: Credit bounds the frames held per connection, not per node: about 1.3 GiB at the connection cap. Advertised credit cannot be taken back, so no hard node-wide bound exists today.

#932
: A reassembled message is re-charged at its full size on the inbound actor, where it cannot wait, and is dropped when that charge does not fit. A chunk that finds the reassembly budget full is dropped too.

#938
: Admission costs $O(n)$ per event and takes node-global locks per frame, exactly when the node is saturated.

#927
: It is unclear whether the barrier's control exemption is still needed under per-lane credit.

Ryan J. Kung can be reached at ryankung@ieee.org. Manuscript created October 25, 2026. This document specifies the load bounds of a Rings node before 1.0: what it holds node-wide, per neighbour edge and per lane, what a free identity can consume, and what refusal and backpressure look like at each bound (rings-transport, module callback; rings-core, modules fair_admission, swarm::callback::inbound, chunk, message::replay and dht::chord::carrier_store). It is the law for issues #907, #927, #932, #934 and #938 of RingsNetwork/rings, and closes the Sybil residual of the removal-stability law [1].

II. Model

Definition 1 (Principals).
- An origin o is an account DID, recovered from a signature. Anyone can create one, at no cost.
- An edge e is a handshake-authenticated connection the node admitted (EdgeRelation::Neighbour). The registry holds at most $\mathcal{R} = 2(F+s+1)$ connection records in any phase, where $F = 160$ is the number of finger slots and s the number of successors (328 at $s = 3$). So at most $\mathcal{R}$ edges are current at any instant. An edge identity is also a free key; what is bounded is the number of edges current at once.
- A lane (e, ℓ) is one of the four credit-controlled channels of an edge.
- The local principal λ is the node itself: its own operations, and deliveries out of its own stores.

Definition 2 (Table). A table T is a node-wide set of items that remote input can grow: replay streams, removal records, frames held, reassembly bytes, mailbox bytes. Every item $x \in T$ carries a principal $\pi(x) \in \mathcal{E} \cup \{\lambda\}$, the edge or local principal that introduced it, and it leaves T only by a transition of T's own law (consumption, retirement at a horizon, or an acknowledged move).

Assumption 1 (What a remote peer can send). A remote party can:

- mint origins without bound;
- hold up to $\mathcal{R}$ edges at once, and replace an edge by a fresh identity whenever connection admission accepts one;
- send, on each edge, anything the edge's credit, quotas and admission laws let through.

It cannot authenticate as an edge it does not hold. Sybil and eclipse resistance remain non-goals; this law bounds what a Sybil consumes, and does not prevent one.

Refusal and backpressure. At a bound, a transition takes one of three forms:

- backpressure: the transition waits for capacity in resource order, and nothing is lost;
- refusal: it fails with an error that names the bound and changes nothing, and the sender keeps its copy;
- drop: it is discarded and counted.

A drop is allowed only where the law names it (best-effort relay, the application inbox). It is never allowed for an honest frame or message at a capacity bound.

III. The accounting law

Law 1 (Edge accounting). Every table T has a per-edge floor f_T, a pool $\mathcal{P}_T$ and a local share L_T, with node-wide bound

$$B_T = \mathcal{R}\, f_T + \mathcal{P}_T + L_T.$$

Let use_e be the number of items introduced by edge e. A transition that adds an item introduced by π is admitted iff:

- $\pi = e$ is current and $\text{use}_e < f_T$ (the edge's floor); or
- the pool has room, $\sum_{e \text{ current}} \max(0, \text{use}_e - f_T) + \text{use}_{\text{departed}} < \mathcal{P}_T$; or
- $\pi = \lambda$ and $\text{use}_\lambda < L_T$.

Otherwise it is refused or, where the law says so, it waits. When an edge departs, its items stay in T and are counted against the pool until they leave by T's own law. No item is ever dropped to make room.

Theorem 1 (Isolation). Under Law 1:

1) $|T| \le B_T$ always;
2) a transition introduced by a current edge e with $\text{use}_e < f_T$ is never refused, however many origins, edges or departed edges any other party controls;
3) a party holding k current edges, and any number of departed ones, occupies at most $k f_T + \mathcal{P}_T$ items.

Proof. (1) At most $\mathcal{R}$ edges are current, each holds at most f_T items within its floor, the pool holds at most $\mathcal{P}_T$, and the local principal holds at most L_T. (2) The floor test reads use_e alone, and only e's own transitions change it. (3) Items within a floor belong to a current edge, and every other item is in the pool. □

Accounting to the origin, as the removal store did [1] and the replay table's quota does, fails part (3). An origin costs nothing, so a party controls unboundedly many, and the only bound left is the node-wide total. The edge is the unit that is both authenticated and bounded in number. The quota per origin stays: it is a rate within an edge, not a capacity. Edge identities are free too, but replacing an edge moves its items to the pool, which Theorem 1 already counts.

IV. Replay streams (#907)

A receiver stream s admits a transaction only while its proof is live, $t < ts + ttl$, with $ttl \le \texttt{MAX_TTL_MS} = H$ and $ts \le t_{\text{admit}} + \sigma$.

Definition 3 (Horizon). Let a_s be the receiver's clock at the last admission of s. The stream is spent at t iff $t \ge a_s + H + \sigma$.

Lemma 1 (Eviction is sound). A spent stream can be forgotten without reopening replay. Every transaction it ever admitted is then expired at every honest clock, so a stream opened afresh at First refuses them as dead proofs.

Proof. Every admitted transaction has $ts \le a_s + \sigma$ and $ttl \le H$, so it expired by $a_s + H + \sigma \le t$. Liveness is checked before replay, so it is refused at the proof, whatever the window holds. □

Law 2 (Stream admission). The receiver table is a table under Law 1:

- A new stream is introduced by the edge that delivered its first transaction, and a relay-inbox delivery by λ.
- A stream leaves the table only when it is spent, by Lemma 1. A spent stream is retired lazily, when a new stream needs its slot.
- The record of a stream stores $(\pi(s), a_s)$, so a restart recovers the charges.

Parameters:

- $f = 32$ new streams per edge, $\mathcal{P} = 4096$ and $L = 1024$, so $B = 328 \cdot 32 + 4096 + 1024 = 15\,616 \le 16\,384$, today's capacity;
- the per-origin quota is unchanged.

A new stream past its bound is refused with TransactionReplayStreamCapacityExceeded, as today. The refusal now lasts until a slot is spent, at most $H + \sigma$ (100 min), and it reaches only the edge past its own floor.

Proposition 1 (No permanent lockout). A party with k edges and any number of origins can refuse new streams only through those k edges, and only once the pool is full. For every other current edge, a new origin is refused only once that edge has itself introduced f unspent streams. Every refusal ends within $H + \sigma$ of the last admission on the streams that fill the bound.

Proof. Theorem 1, and Lemma 1 for the end of a refusal. □

A relay or seed that forwards many distinct origins can fill its own floor legitimately. Its overflow then competes in the pool; the honest cost is a refusal for at most $H + \sigma$ on that edge alone. The sender table holds this node's own streams, so it is local.

V. The removal store, re-keyed

Law 6 of [1] charged each remove to its sender's origin. We re-key it under Law 1:

- an operation or a sync is charged to the edge that delivered it, and this node's own writes and repairs to λ;

- the holder class goes away: a predecessor or successor is an edge like any other, so its floor is the same.

Parameters: $f = 128$ removes per edge, $\mathcal{P} = 16\,384$ and $L = 7\,168$, so $B = 328 \cdot 128 + 16\,384 + 7\,168 = 65\,536$, today's total. Theorem 1 then closes the Sybil residual that #907 was to close: one mechanism serves both tables.

VI. Received frames (#934)

Let lane (e, ℓ) have advertised credit A and released count R. Its liability is liab $= A - R$: the frames the receiver may still have to hold, in flight or held. Today liab $\leq W = 16$ per lane, but credit is cumulative and never shrinks, so $\sum$ liab is bounded only by the connection count.

Definition 4 (Reclaim). Two frames are added on the priority lane, both cumulative and idempotent:

- $\mathsf{Shrink}(\ell, L', g)$, receiver to sender: "from generation g, your limit on lane ℓ is L'".
- $\mathsf{Returned}(\ell, c, g)$, sender to receiver: "I committed c frames of ℓ, and from now on I commit at most $\max(c, L')$".

Until the Returned arrives, the receiver counts the lane's liability at its old A. After it, the liability is $\max(c, L') - R$.

Law 3 (Hard frame bound). Every lane has a floor g_ℓ: $g_{\text{prio}} = 2$ for the priority lane (control, and the reclaim and demand frames), and $g_\ell = 0$ for the data lanes. A fresh generation grants only the floors.

- A data lane with queued frames and no credit sends a one-frame $\mathsf{Wanted}(\ell)$ on the priority lane.
- The receiver grants a lane credit beyond its floor only from the node's frame pool:

$$\sum \text{liab} \leq B_F = \mathcal{R}\, g_{\text{prio}} + \mathcal{P}_F.$$

- Demands are served first come, first served.
- A lane that received no frame for T_{idle} while holding pooled credit is shrunk to its floor.

Parameters:

- $\mathcal{P}_F = 256$ frames (16 MiB, the soft limit of today);
- T_{idle} = two observed round trips;
- $B_F = 328 \cdot 2 + 256 = 912$ frames, 57 MiB.

This is independent of how many connections are open.

Proposition 2 (Frames). Under Law 3, for an honest sender:

1) $\sum$ liab $\leq B_F$ in every interleaving, with shrink, return and demand frames reordered and duplicated;
2) no honest frame is refused: a frame sent under credit is within the liability counted for it;
3) a lane with queued frames is never blocked forever while the receiver keeps consuming (progress);
4) an idle lane holds at most its floor after T_{idle} plus one round trip, so it starves no active lane;
5) control traffic never waits for the pool.

Proof sketch. (1) Pooled credit is granted only when the counted liability fits, and the counted liability never falls below the true one: a shrink takes effect only once its Returned confirms what was committed. (2) A frame committed under A is counted until released. (3) A blocked lane has a Wanted queued. The pool refills as frames are released and idle lanes are shrunk, and service is first come, first served. (4) The shrink frees the lane's pooled credit. (5) The priority floor is reserved per edge. These are the model-checking obligations of the implementation, over the existing credit model with the new frames. □

This is a wire change: a total cutover, with no versioning before 1.0. Byte-denominated credit (#937) composes with it, by counting liab in bytes, and is not required here.

VII. Reassembled messages (#932)

Law 4 (One charge per message). The first chunk of a message, with header $[0, n]$ and data length d_0, reserves $m = n \cdot d_0$ (validated $\leq$ max_message_bytes) once, on the inbound mailbox ledger: the edge's budget, the lane's share and the shared pool. It takes that reservation as an arrival, so it waits in resource order and is never refused for capacity.

- Later chunks of the message must have length $\leq d_0$, and they charge nothing.
- The reassembly buffer and the completed message live inside that one reservation.
- The mailbox transition at completion is a move, not a new charge.
- The reservation is released when the message's processing completes, or when the reassembly expires.

The separate reassembly byte budget and its refusal (Rejected(Capacity)) are retired. The id and slot caps stay, as validity bounds.

Proposition 3 (No honest message refused). Under Law 4, no honest chunked message is refused or dropped for capacity, and no wait can deadlock.

Proof. The only charge is the first chunk's, which waits as an arrival before it holds a lane ticket. The ordering laws of ResourceOrderedQueue cover it, and the capacity it waits for is released by events that do not wait on it. The transition that used to re-charge, and could only be refused, now moves bytes already reserved. □

The cost is direction 1 of #932: a message holds its full size for its whole reassembly. This is bounded by the edge's budget and by the reassembly expiry, so a peer that announces and then stalls holds only its own budget.

VIII. Hot paths (#938)

These are performance obligations, kept apart from the security laws above.

Obligation 1 (Admission complexity). • Each arrival, release and cancel in ResourceOrderedQueue costs $O(\log n)$ in the waiters: waiters are indexed by resource, and one ledger snapshot is taken per pass.

- Each received frame takes $O(1)$ lock acquisitions, and none of them on a node-global mutex: the node load is an atomic counter.
- The node load has hysteresis: advertisements are deferred above H_{hi} and resume below $H_{\mathrm{lo}} = H_{\mathrm{hi}}/2$. Deferred links are kept in a set, so a release at the limit wakes no one, and crossing H_{lo} wakes each deferred link once.
- Each peer runs one admission watcher, not one task per send.
- Unread admissions are indexed by peer.

They are verified against the sync-storm matrix, before and after each change.

IX. The barrier exemption (#927)

Lemma 2 (One frame per lane). If every path that hands a frame to core does so through its lane's FIFO, with at most one frame in hand-over per lane, then a lane that waits behind the reassembly barrier waits at most one reassembly frame's service, $\leq 2\,\mathrm{s}$. That is below the control deadline of 15 s.

The predicate lane_waits_for_reassembly is "the lane carries logical data". Chunks travel only on their message's lane (the lane-mismatch refusal), and control messages are never chunked, so the predicate is exactly "the lane can carry chunks". In that form it is the barrier's definition, not an exemption.

Decision. Keep the predicate, renamed for what it is. Remove the ablation toggle ProtectionLayer::BarrierControlExemption and its legacy feedback-loop witness. This is a subtraction, under two obligations checked in the implementation:

- every hand-off path obeys Lemma 2, the pre-admission drain and the detached submissions included;
- no control message can exceed one frame. This is asserted at compile time from the widest control message.

X. Summary of bounds

Table	Bound	At the bound
frames held	node B_F; lane floor	sender waits for credit (backpressure)
mailbox bytes	edge budget, lane share, pool	arrival waits (backpressure)
reassembly	inside the mailbox reservation	first chunk waits (backpressure)
replay streams	$\mathcal{R}f + \mathcal{P} + L$ by edge	new stream refused, $\leq H + \sigma$
removal store origin	$\mathcal{R}f + \mathcal{P} + L$ by edge rate quota within its edge	write refused, unacked refused for the rate window
application inbox	64 MiB	dropped, counted (by design)

What a free identity consumes. An origin alone consumes nothing beyond its quota on the edge that carries it. An edge consumes its floors. The pool is shared, and is the most that any number of free identities can take together.

XI. Implementation order

PR	Issue	Law
1	#907	Laws 1, 2; Sec. V
2	#932	Law 4
3	#934	Law 3 (wire cutover)
4	#938	Obligation 1, measured
5	#927	Sec. IX (subtraction)

PR 1 adds the edge-accounting primitive that PR 2 reuses for the mailbox. PR 4 rewrites the queue that PR 2 changes, so it follows PR 2. PR 5 needs nothing from the others.

Operator steps, in the CHANGELOG under Unreleased:

- PR 1 changes the replay record, which gains its introducer and last admission. Remove the transaction-replay store while the node is stopped; this is the documented reset of the durable-records law. The removal store gains an edge charge, so remove dht-removals too.
- PR 3 is a wire cutover with no step.

References

[1] R. J. Kung, "Removal stability: Eviction, tombstone bounds and heartbeats for the data topics of Rings," papers/removal-stability.tex, https://github.com/RingsNetwork/rings, 2026, issue #1060.